Library System using CI/CD Pipeline with Docker and Kubernetes

Course name: Agile Development Process and Devops
Course Code: ISWE406L

Submitted By:

Arunima Sanjeev (23MIS0048)

ABSTRACT

This project focuses on the development and deployment of a Library Management System using modern DevOps practices and tools. The application was designed in three progressive versions to demonstrate incremental software development. Version 1 implements basic functionality for storing and retrieving book data, Version 2 introduces the ability to issue books to students, and Version 3 completes the system by enabling the return of issued books, thereby achieving a complete lifecycle.

The application was developed using Visual Studio Code (VS Code) with Python and the Flask framework, while version control was managed using GitHub. Separate commits were created for each version (v1, v2, v3) to track incremental development. A Continuous Integration and Continuous Deployment (CI/CD) pipeline was implemented using Jenkins to automate the build and deployment process. Each version of the application was containerized using Docker, ensuring consistency across environments, and the images were stored in DockerHub for centralized access.

For deployment, Kubernetes was used to manage and run the application. Each version was deployed as a separate service using NodePort, allowing access through different ports on the local system. The project successfully demonstrates the integration of development and operations, highlighting the importance of automation, scalability, and efficient deployment strategies in modern software systems.

INTRODUCTION

In modern software development, the need for faster delivery, reliability, and scalability has led to the adoption of DevOps practices. DevOps is a methodology that integrates development and operations to streamline the software lifecycle through automation and continuous monitoring. Two key components of DevOps are Continuous Integration (CI) and Continuous Deployment (CD), which enable automatic building, testing, and deployment of applications.

This project focuses on developing a Library Management System while implementing a complete CI/CD pipeline using industry-standard tools. The application was designed in three progressive versions to simulate real-world software evolution. Version 1 provides basic functionality to store and retrieve book data, Version 2 introduces the ability to issue books to students, and Version 3 completes the system by adding a return mechanism, thereby achieving a full operational lifecycle.

The backend of the application was developed using Python and the Flask framework in Visual Studio Code (VS Code), which provided an efficient development environment for building REST APIs. Version control was maintained using GitHub, where separate commits were created for each version (v1, v2, v3) to ensure proper tracking of incremental development and changes.

For deployment, Kubernetes was used to manage and run the application as separate deployments and services. This allowed each version to be accessed independently through different ports on the local system. The project demonstrates how modern DevOps tools can be integrated to build, deploy, and manage applications efficiently, while also highlighting the importance of automation, scalability, and modular development.

SYSTEM ARCHITECTURE

The system follows a DevOps pipeline architecture:

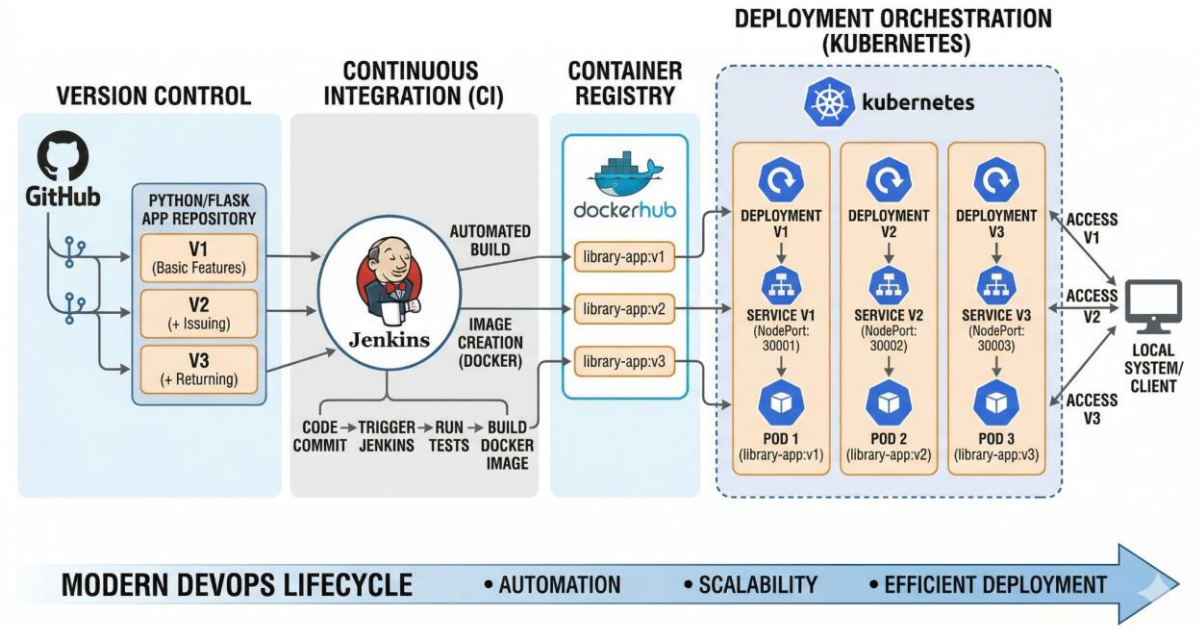


Figure 1: System Architecture of the Project

- Source code is stored and managed in GitHub
- Jenkins pulls the latest code and executes the CI/CD pipeline
- Docker is used to build container images for each version
- DockerHub stores and manages the built images
- Kubernetes deploys and exposes the application using services

The architecture follows a modular and scalable DevOps pipeline where each component plays a specific role in the software lifecycle. GitHub serves as the version control system, enabling efficient code management and collaboration. Jenkins automates the build and integration process, ensuring that any changes in the code are automatically processed. Docker is used to containerize the application, providing consistency across development and deployment environments. DockerHub acts as a centralized image repository, while Kubernetes handles orchestration, deployment, and service exposure. This architecture ensures automation, scalability, and reliability of the application.

IMPLEMENTATION

6.1 Project Setup

The project structure was designed to maintain modularity and clarity. Each version was developed in a separate folder to simulate real-world software versioning and incremental feature addition. This approach allows independent development, testing, and deployment of each version without affecting others. The use of a structured directory layout also simplifies integration with CI/CD tools such as Jenkins.

A project folder named library-system was created. Inside this folder, three subfolders were created:

- v1
- v2
- v3

Each folder represents a version of the application.

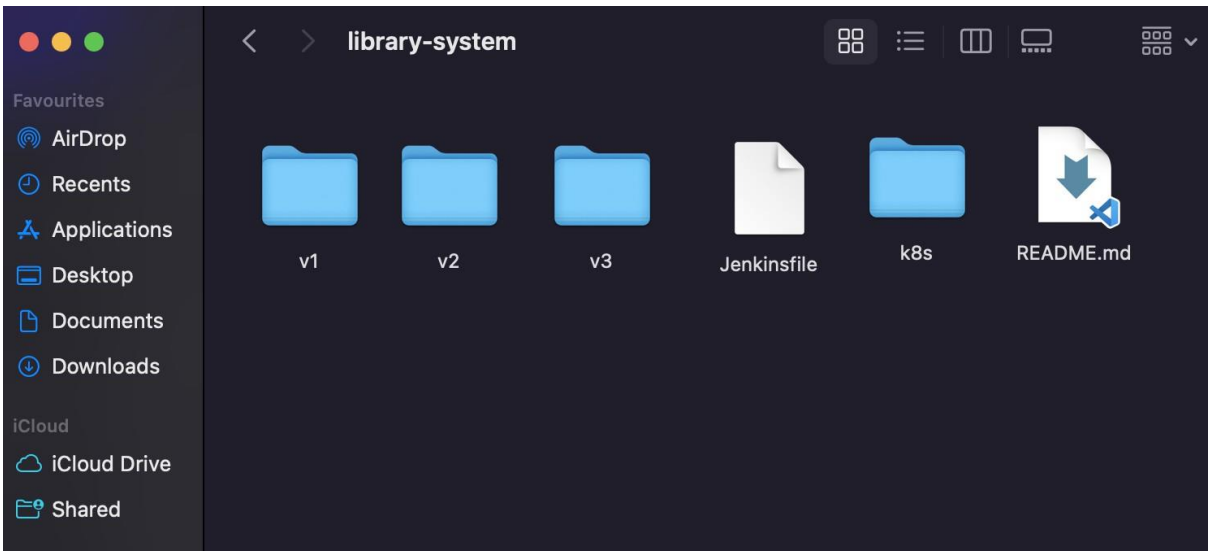


Figure 2: Project Folder Structure

6.2 Version 1 – Add Book

Version 1 implements the basic functionality of adding books to the system. The application provides APIs to add books and view all books.

Features:

- Add book using /add_book
- View books using /books

Version 1 serves as the foundation of the application by implementing core functionalities for managing book data. It uses RESTful APIs to allow interaction with the system. The data is stored in-memory using a list structure, which simplifies implementation and testing. This

version establishes the base upon which further functionalities are built in subsequent versions.

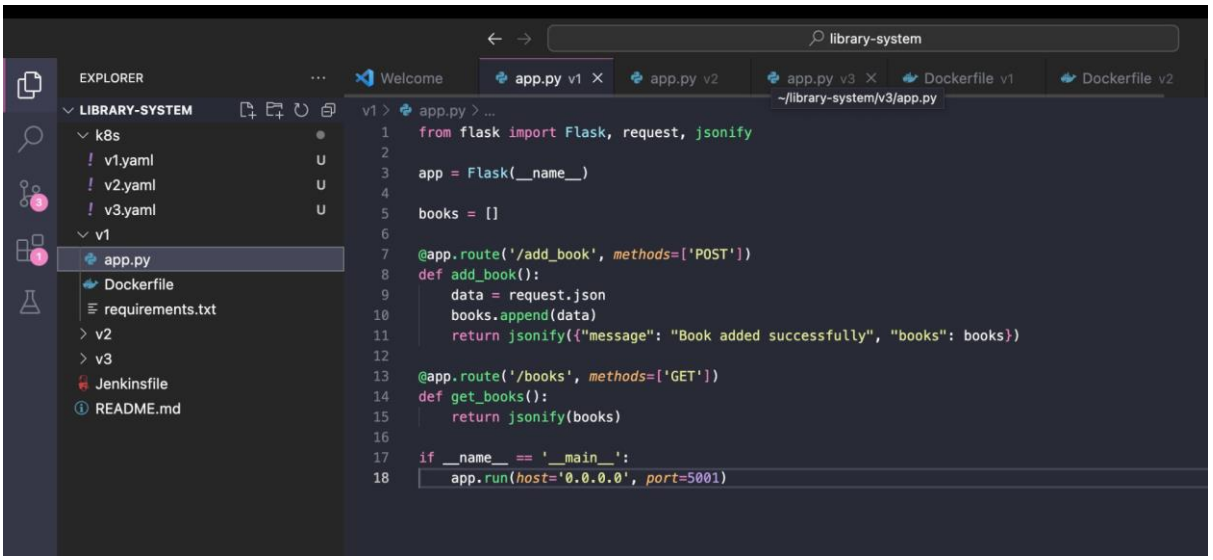


Figure 3: Version 1 Code Implementation

Figure 4: Adding books using /add_book API in Version 1

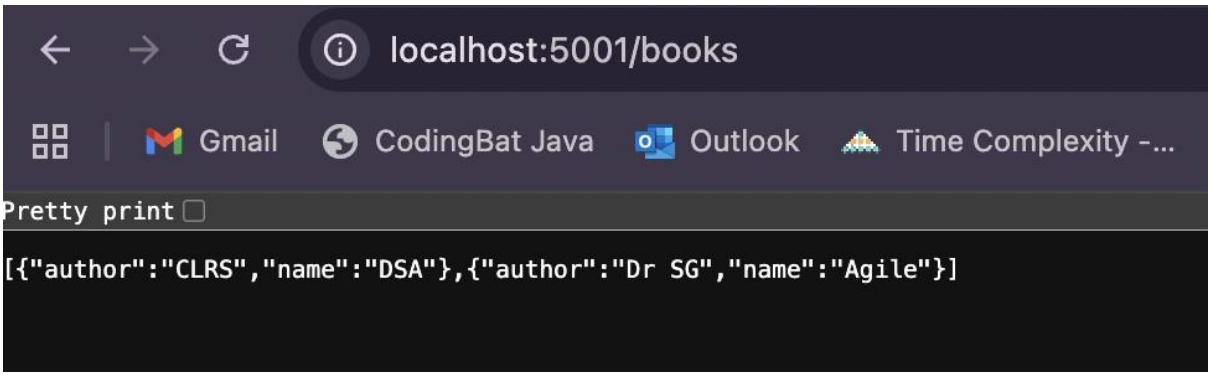


Figure 5: Viewing stored books using /books endpoint in Version 1

6.3 Version 2 – Issue Book

Version 2 extends the functionality by allowing books to be issued to students. A dictionary was used to store issued books.

Features:

- Issue book using /issue
- Validation for book existence

- Prevent duplicate issuing

Version 2 enhances the system by introducing the concept of book issuance. A dictionary data structure is used to map books to students, enabling efficient tracking of issued books. Additional validation checks ensure data integrity, such as verifying book availability and preventing duplicate issuance. This version demonstrates the transition from basic data handling to functional system logic.

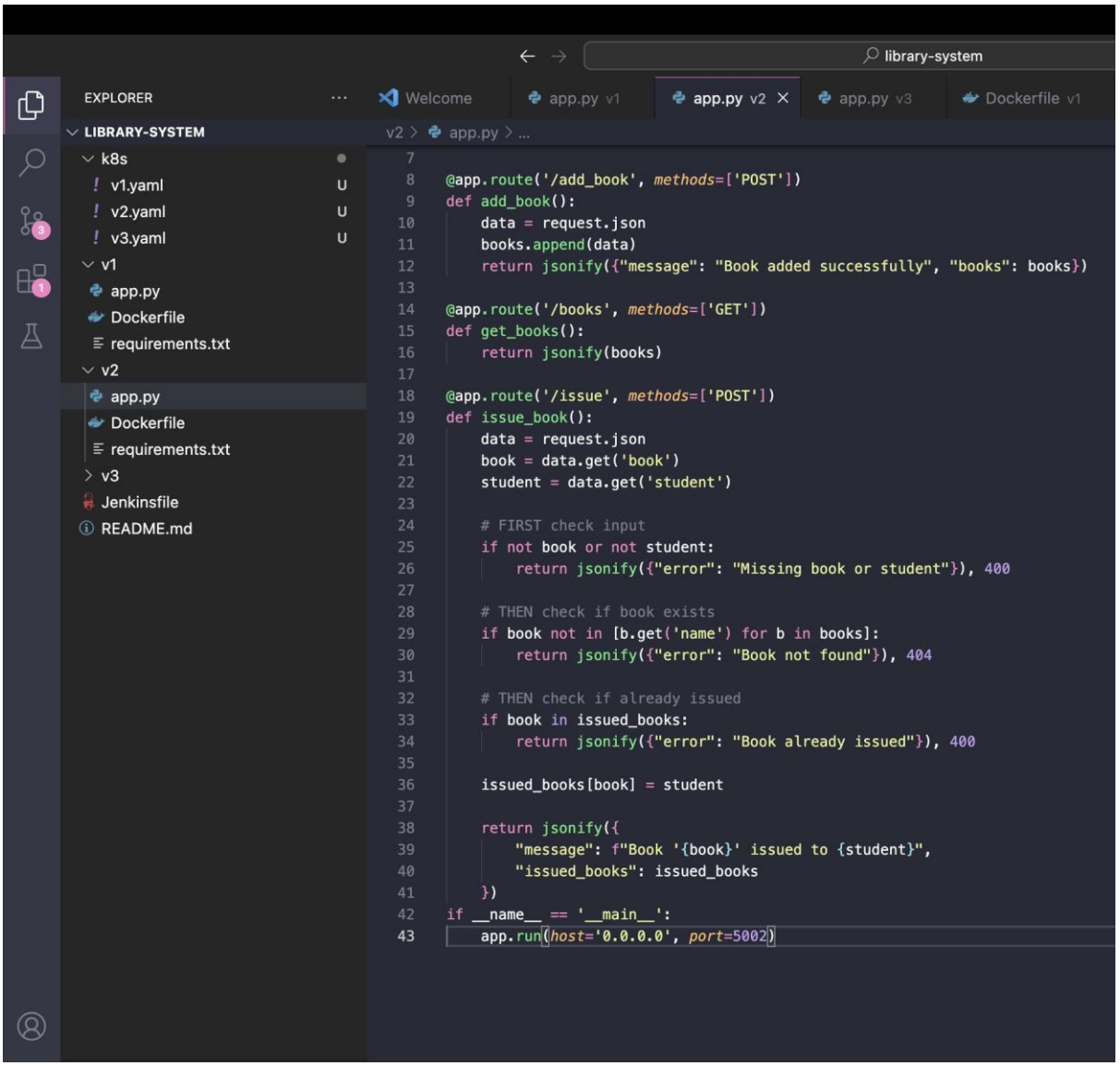


Figure 6: Version 2 Code Implementation

```
arunimasanjeev@Arunimas-MacBook-Air ~ % curl -X POST http://localhost:5003/add_book \
-H "Content-Type: application/json" \
-d '{"name":"DSA","author":"CLRS"}'
{"books":[{"author":"CLRS","name":"DSA"}],"message":"Book added successfully"}
arunimasanjeev@Arunimas-MacBook-Air ~ % curl -X POST http://localhost:5003/issue \
-H "Content-Type: application/json" \
-d '{"book":"DSA","student":"John"}'
{"issued_books":{"DSA":"John"},"message":"Book 'DSA' issued to John"}
```

Figure 7: Execution of add and issue APIs in Version 2

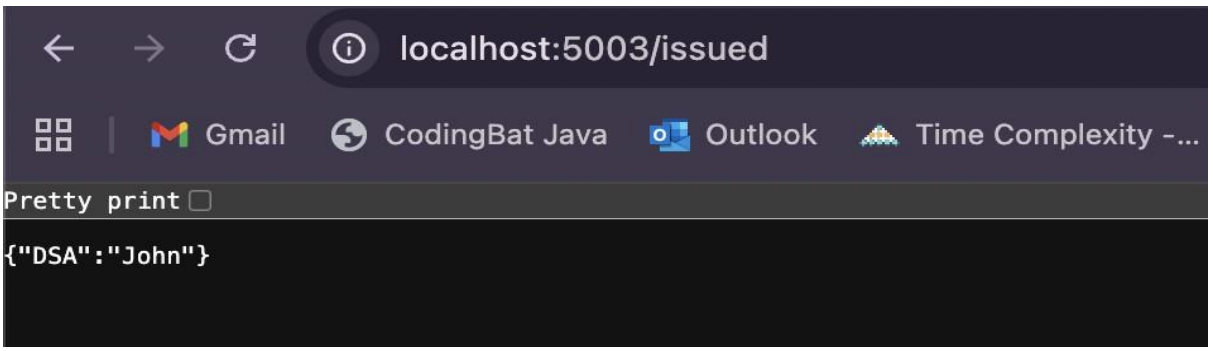


Figure 8: Display of issued books using /issued endpoint in Version 2

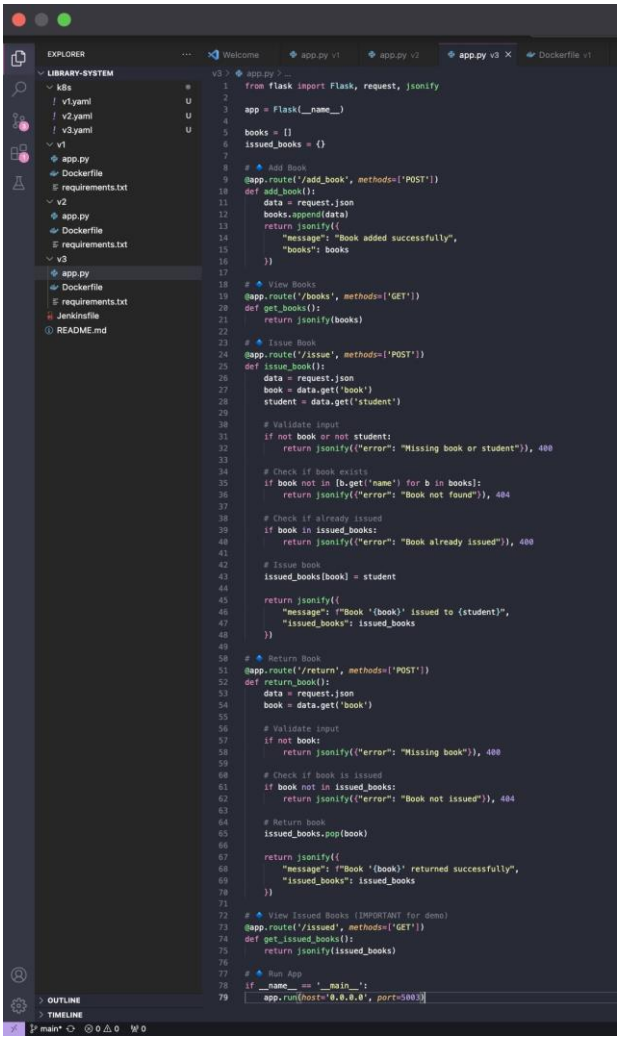
6.4 Version 3 – Return Book

Version 3 completes the lifecycle by adding the return functionality.

Features:

- Return book using /return
- View issued books using /issued
- Complete lifecycle: Add → Issue → Return

Version 3 completes the system by implementing the return functionality, thereby achieving a full lifecycle of operations. The return feature updates the system state by removing issued books from the tracking structure. This version demonstrates a complete workflow, making the system more practical and closer to real-world applications. It highlights how incremental development improves system capabilities over time.



```python
from flask import Flask, request, jsonify

app = Flask(__name__)

books = []
issued_books = {}

# 📗 Add Book
@app.route('/add_book', methods=['POST'])
def add_book():
    data = request.json
    books.append(data)
    return jsonify({
        "message": "Book added successfully",
        "books": books
    })

# 📘 View Books
@app.route('/books', methods=['GET'])
def get_books():
    return jsonify(books)

# 📤 Issue Book
@app.route('/issue', methods=['POST'])
def issue_book():
    data = request.json
    book = data.get('book')
    student = data.get('student')

    # Validate input
    if not book or not student:
        return jsonify({"error": "Missing book or student"}), 400

    # Check if book exists
    if book not in [b.get('name') for b in books]:
        return jsonify({"error": "Book not found"}), 404

    # Check if already issued
    if book in issued_books:
        return jsonify({"error": "Book already issued"}), 400

    # Issue book
    issued_books[book] = student

    return jsonify({
        "message": f"Book '{book}' issued to {student}",
        "issued_books": issued_books
    })

# 📥 Return Book
@app.route('/return', methods=['POST'])
def return_book():
    data = request.json
    book = data.get('book')

    # Validate input
    if not book:
        return jsonify({"error": "Missing book"}), 400

    # Check if book is issued
    if book not in issued_books:
        return jsonify({"error": "Book not issued"}), 404

    # Return book
    issued_books.pop(book)

    return jsonify({
        "message": f"Book '{book}' returned successfully",
        "issued_books": issued_books
    })

# 📋 View Issued Books (IMPORTANT for demo)
@app.route('/issued', methods=['GET'])
def get_issued_books():
    return jsonify(issued_books)

# 🚀 Run App
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5003)
```

Figure 9: Version 3 Code Implementation

```
arunimasanjeev@Arunimas-MacBook-Air ~ % curl -X POST http://localhost:5003/add_book \
-H "Content-Type: application/json" \
-d '{"name":"DSA","author":"CLRS"}'
{"books":[{"author":"CLRS","name":"DSA"},{"author":"CLRS","name":"DSA"}],"message":"Book added successfully"}
arunimasanjeev@Arunimas-MacBook-Air ~ % curl -X POST http://localhost:5003/issue \
-H "Content-Type: application/json" \
-d '{"book":"DSA","student":"John"}'
{"error":"Book already issued"}
arunimasanjeev@Arunimas-MacBook-Air ~ % curl -X POST http://localhost:5003/return \
-H "Content-Type: application/json" \
-d '{"book":"DSA"}'
{"issued_books":{},"message":"Book 'DSA' returned successfully"}
```

Figure 10: Execution of add, issue and return APIs in Version 3

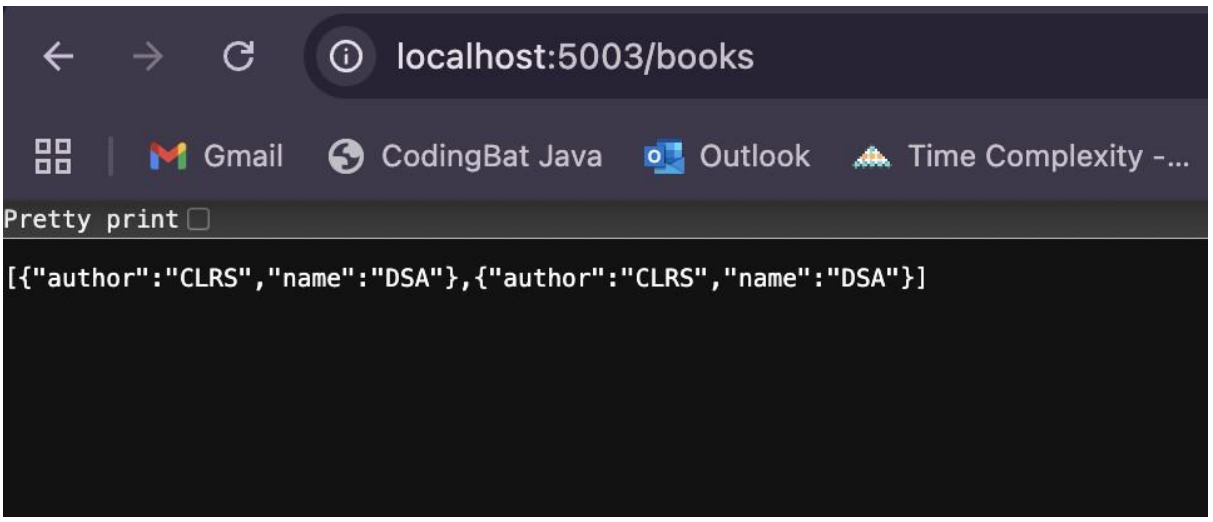


Figure 11: Viewing stored books using /books endpoint in Version 3

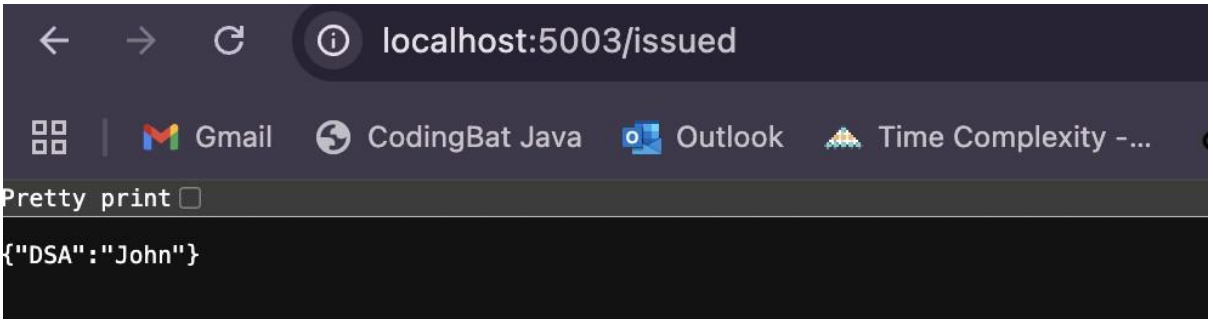


Figure 12: Display of issued books using /issued endpoint in Version 3

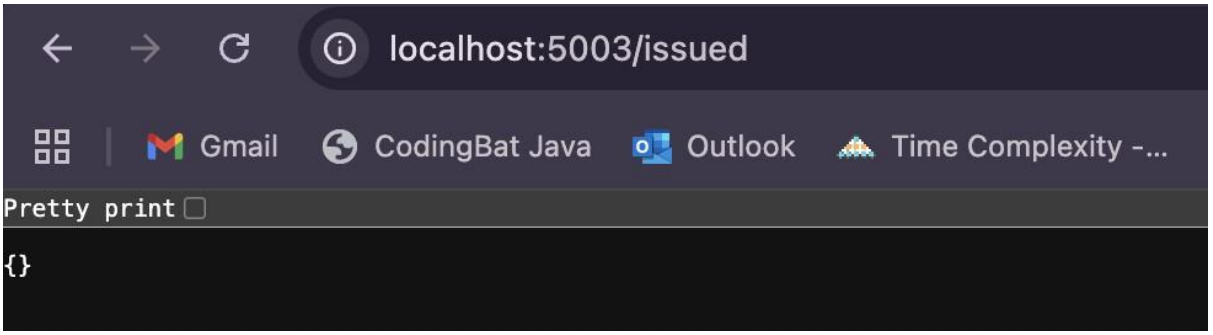


Figure 13: Display of issued books after return using /issued endpoint in Version 3

GITHUB INTEGRATION

The project was uploaded to GitHub for version control and integration with Jenkins. Separate commits were created for each version (v1, v2, v3) to track incremental development. GitHub was used as a version control platform to manage the project code efficiently. It enabled tracking of changes, maintaining version history, and facilitating integration with Jenkins for automated workflows. The use of Git commands ensured proper versioning and helped in maintaining a structured development process.

Steps:

- Initialized Git repository
- Added project files
- Pushed to GitHub

```
arunimasanjeev@Arunimas-MacBook-Air library-system % git init
Initialized empty Git repository in /Users/arunimasanjeev/library-system/.git/
arunimasanjeev@Arunimas-MacBook-Air library-system % git add .
arunimasanjeev@Arunimas-MacBook-Air library-system % git commit -m "final project"
[main (root-commit) 0d83427] final project
 11 files changed, 170 insertions(+)
 create mode 100644 Jenkinsfile
 create mode 100644 README.md
 create mode 100644 v1/Dockerfile
 create mode 100644 v1/app.py
 create mode 100644 v1/requirements.txt
 create mode 100644 v2/Dockerfile
 create mode 100644 v2/app.py
 create mode 100644 v2/requirements.txt
 create mode 100644 v3/Dockerfile
 create mode 100644 v3/app.py
 create mode 100644 v3/requirements.txt
arunimasanjeev@Arunimas-MacBook-Air library-system % git remote add origin https://github.com/ArunimaSanjeev/library-system
arunimasanjeev@Arunimas-MacBook-Air library-system % git push -u origin main
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 8 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (12/12), 1.51 KiB | 1.51 MiB/s, done.
Total 12 (delta 3), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (3/3), done.
To https://github.com/ArunimaSanjeev/library-system
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Figure 14: Commands used to upload project to GitHub

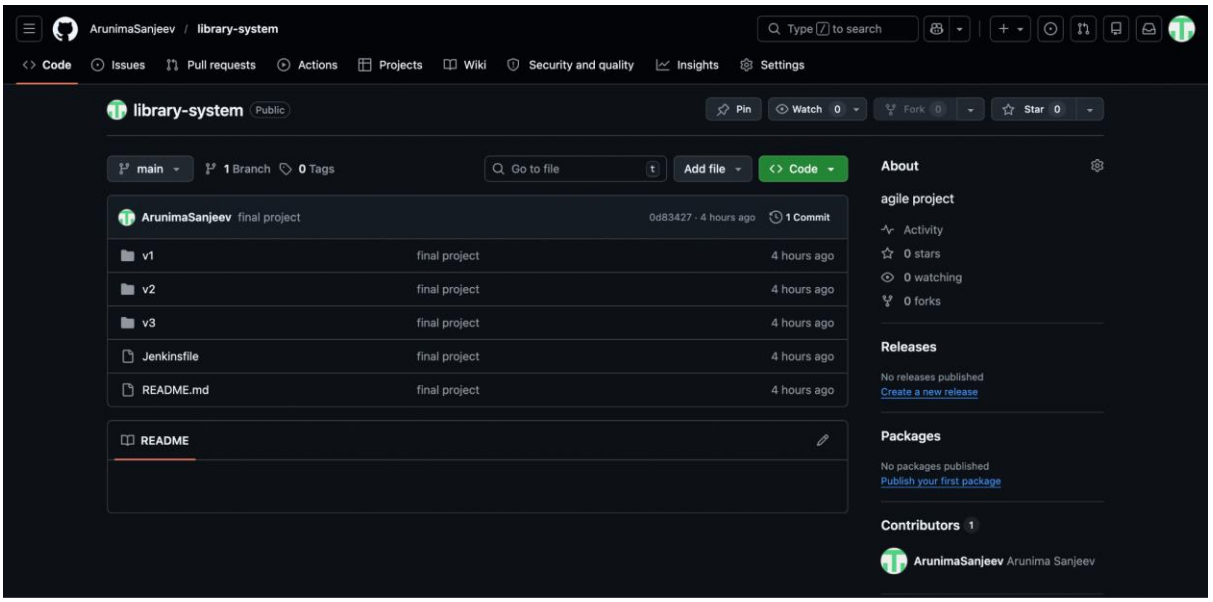


Figure 15: GitHub repository

DOCKER IMPLEMENTATION

Docker was used to containerize the application, ensuring that it runs consistently across different environments. By packaging the application along with its dependencies, Docker eliminates compatibility issues. Each version of the application was built as a separate

image, allowing independent deployment and testing. This approach enhances portability and simplifies deployment.

Steps:

- Created Dockerfile
- Built images using docker build
- Ran containers locally

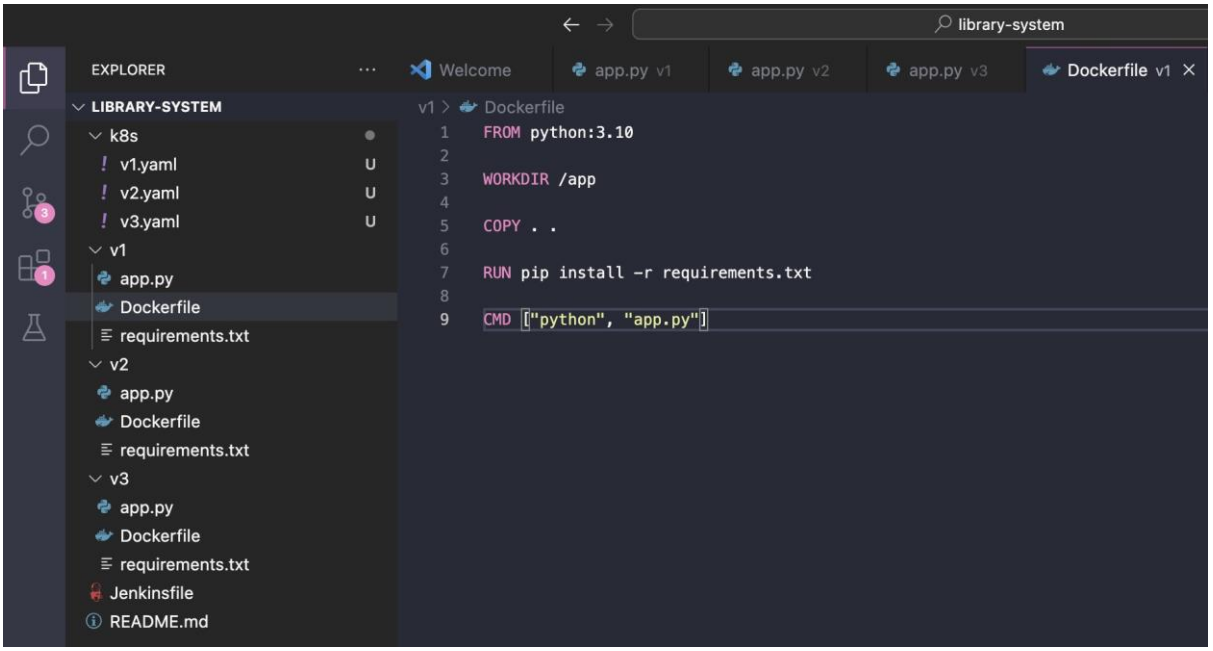


Figure 16: Dockerfile

```
arunimasanjeev@Arunimas-MacBook-Air library-system % docker build -t yourdockerhub/library:v1 ./v1
docker build -t yourdockerhub/library:v2 ./v2
docker build -t yourdockerhub/library:v3 ./v3
[+] Building 820.1s (10/10) FINISHED
```

Figure 17: Built images using docker build

```
arunimasanjeev@Arunimas-MacBook-Air library-system % docker ps
CONTAINER ID   IMAGE                      COMMAND           CREATED          STATUS          PORTS                    NAMES
d0dcbe39d641   yourdockerhub/library:v3   "python app.py"   10 seconds ago   Up 9 seconds    0.0.0.0:5003->5003/tcp   youthful_wozniak
f67998214bbb   yourdockerhub/library:v2   "python app.py"   10 seconds ago   Up 10 seconds   0.0.0.0:5002->5002/tcp   angry_vaughan
fe3f9a5d0e1f   yourdockerhub/library:v1   "python app.py"   11 seconds ago   Up 10 seconds   0.0.0.0:5001->5001/tcp   funny_agnesi
```

Figure 18: Running Docker containers for Version 1, Version 2, and Version 3 applications, showing active status and port mappings

DockerHub Integration

Docker images were pushed to DockerHub. DockerHub was used as a centralized repository to store and manage Docker images. After building the images locally, they were pushed to DockerHub, making them accessible for deployment in Kubernetes. This step is essential in CI/CD pipelines as it allows seamless transfer of application images between environments.

Images:

- library:v1
- library:v2
- library:v3

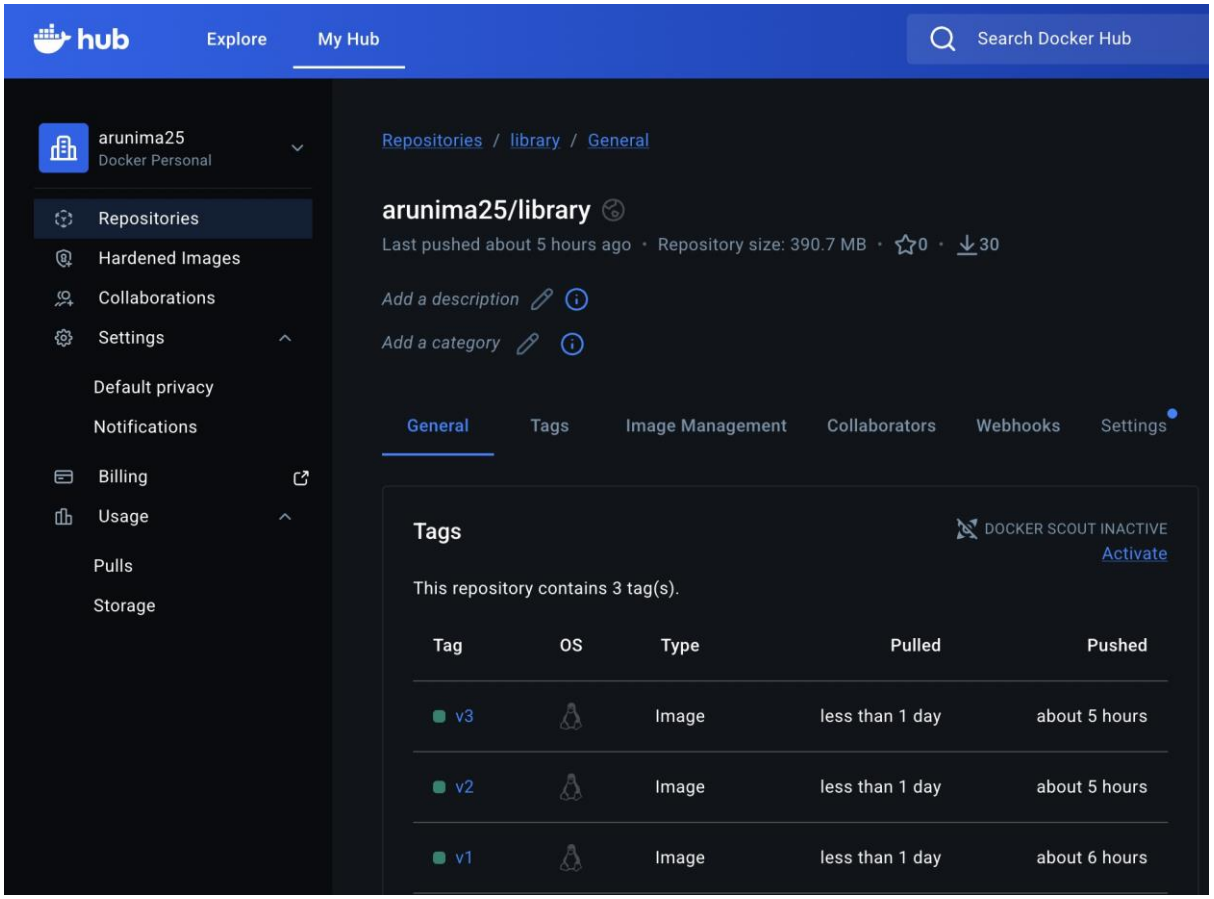


Figure 19: DockerHub Repository

JENKINS PIPELINE

A CI/CD pipeline was created inside Jenkins to automate the build and push process.

Pipeline Stages:

- Clone Repository
- Build Docker Images
- Push Images to DockerHub

Jenkins was used to automate the CI/CD pipeline by integrating code from GitHub, building Docker images, and pushing them to DockerHub. The pipeline was divided into multiple stages, each responsible for a specific task. This automation reduces manual effort, minimizes errors, and ensures faster deployment. It demonstrates the importance of continuous integration in modern software development.

Jenkins / All / New Item

New Item

Enter an item name

library-pipeline

Select an item type

Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.

Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

Maven project
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.

Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

Folder
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

Multibranch Pipeline
Creates a set of Pipeline projects according to detected branches in one SCM repository.

Organization Folder
Creates a set of multibranch project subfolders by scanning for repositories.

If you want to create a new item from other existing, you can use this option:

Copy from

Type to autocomplete

OK

localhost:8080

Figure 20: Jenkins Pipeline Creation

Jenkins / library-pipeline / Configure

Configure

- General
- Triggers
- Pipeline
- Advanced

Poll SCM ?

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script

Script ?

try sample Pipeline...

```
pipeline {
    agent any

    stages {

        stage('Clone Repo') {
            steps {
                git 'https://github.com/ArunimaSanjeev/library-system.git'
            }
        }

        stage('Build v1') {
            steps {
                sh 'docker build -t arunima25/library:v1 ./v1'
            }
```

Use Groovy Sandbox ?

Pipeline Syntax

Advanced

Save Apply

Figure 21: Jenkins Pipeline Script

```
40d285db816b: Waiting
86cdc6c7463a: Waiting
f94f601d560a: Waiting
40d285db816b: Waiting
d16813bdedcf: Waiting
6778b62bd97b: Waiting
47a664926042: Waiting
0cbad1ea40b8: Waiting
f4a64930b7a2: Waiting
028f748cb5ee: Waiting
0cbad1ea40b8: Layer already exists
f4a64930b7a2: Layer already exists
028f748cb5ee: Waiting
47a664926042: Waiting
f94f601d560a: Layer already exists
40d285db816b: Waiting
d16813bdedcf: Layer already exists
6778b62bd97b: Layer already exists
86cdc6c7463a: Layer already exists
40d285db816b: Waiting
028f748cb5ee: Waiting
40d285db816b: Waiting
028f748cb5ee: Waiting
40d285db816b: Waiting
028f748cb5ee: Pushed
40d285db816b: Pushed
47a664926042: Pushed
v3: digest: sha256:b03c3198ecaedbe0ccb379962ff9ba7008cd9b094a5a317e47544a48c55cb8c6 size: 856
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Figure 22: Jenkins Pipeline Execution

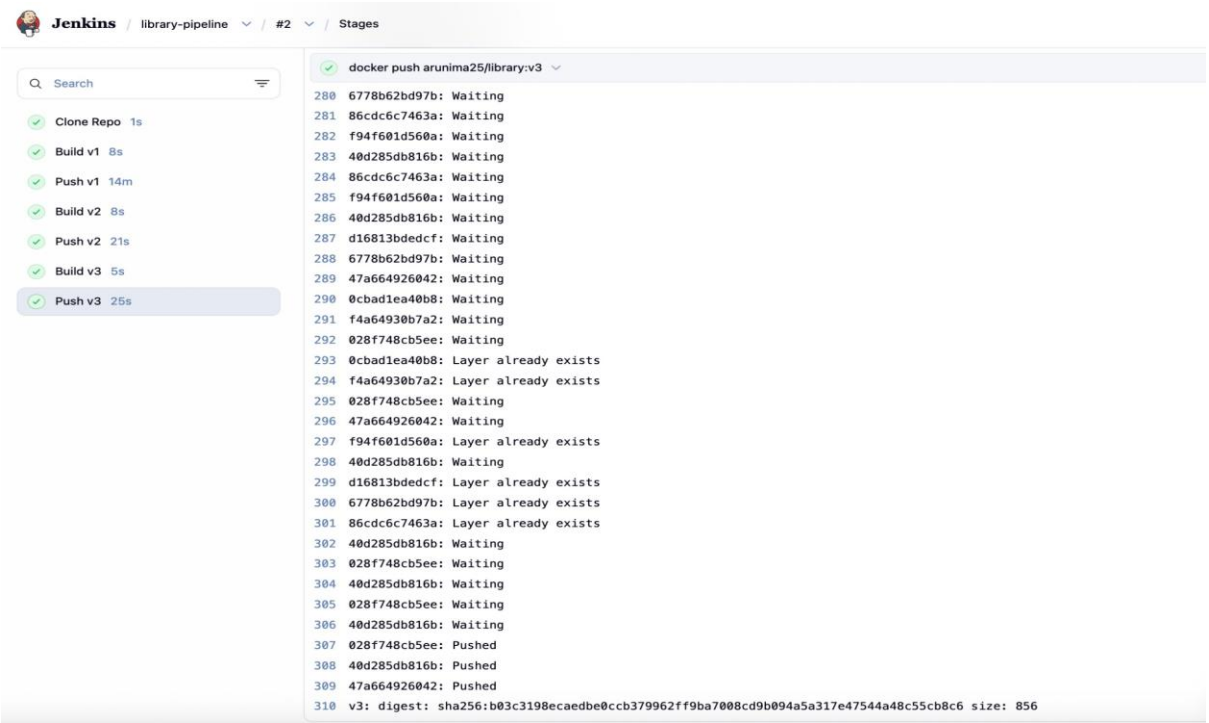


Figure 23: Jenkins Pipeline Stages

KUBERNETES DEPLOYMENT

Kubernetes was used to deploy the application.

Steps:

- Created YAML files for v1, v2, v3
- Created deployments and services
- Used NodePort to expose applications

Kubernetes was used to deploy and manage containerized applications. Each version was deployed as a separate deployment and exposed using NodePort services. This allows users to access different versions independently through specific ports. Kubernetes ensures scalability, fault tolerance, and efficient resource management, making it suitable for production-level deployments.

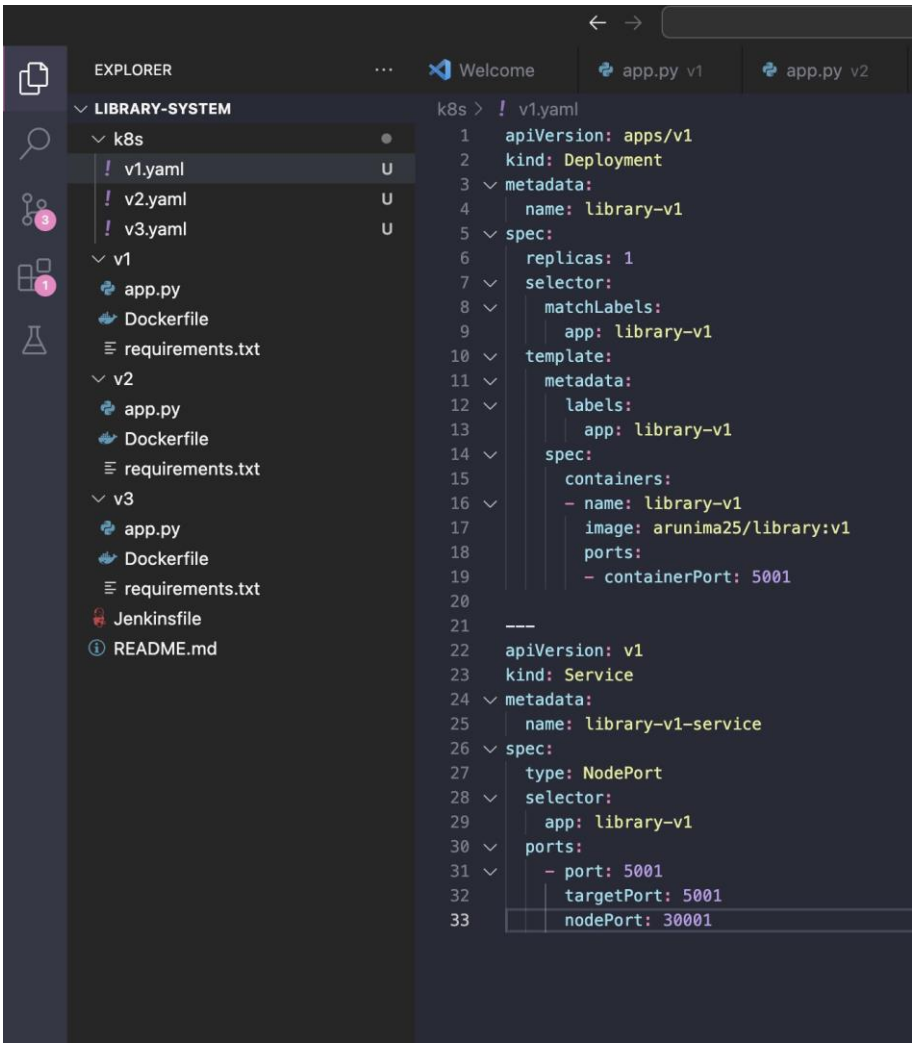


Figure 24: Kubernetes Configuration for v1

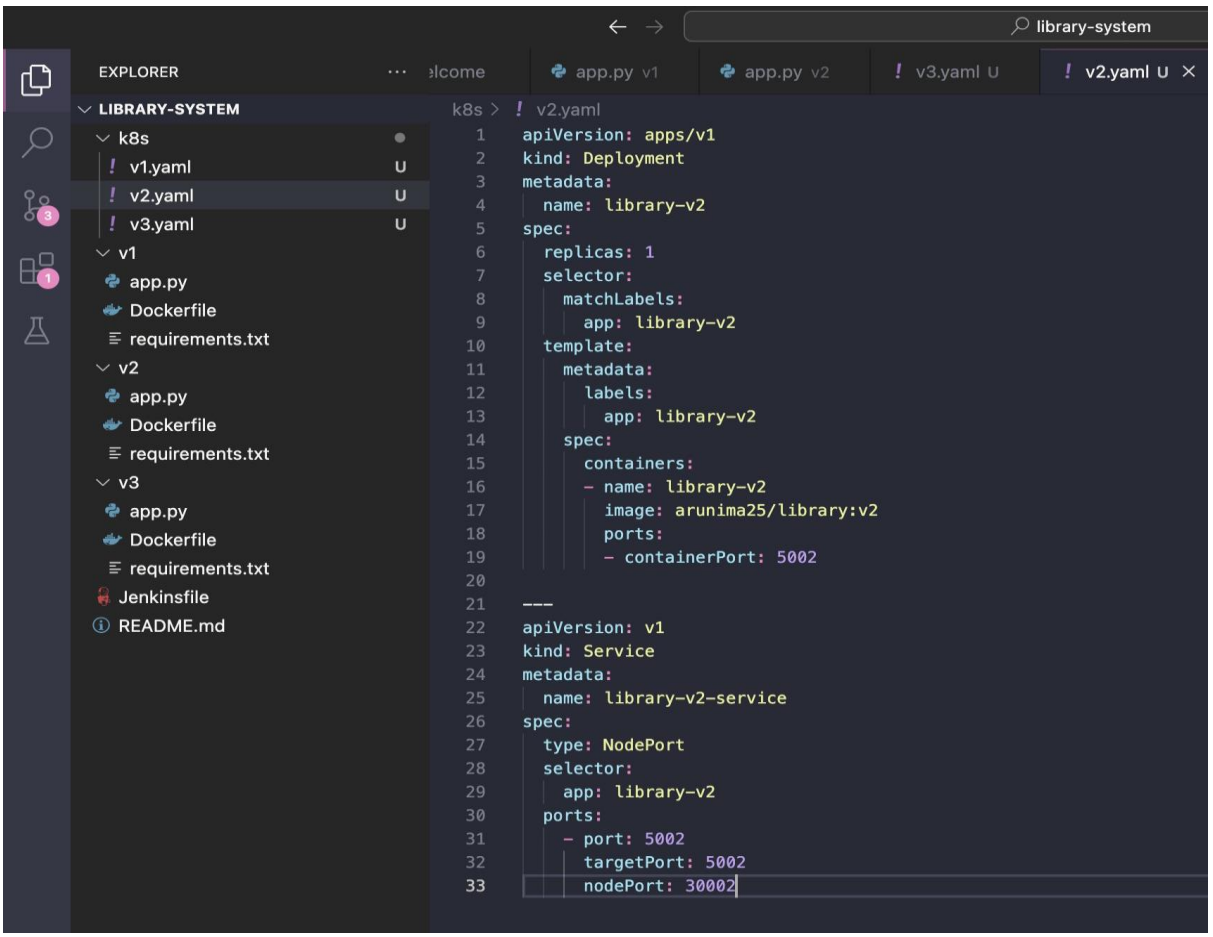


Figure 25: Kubernetes Configuration for v2

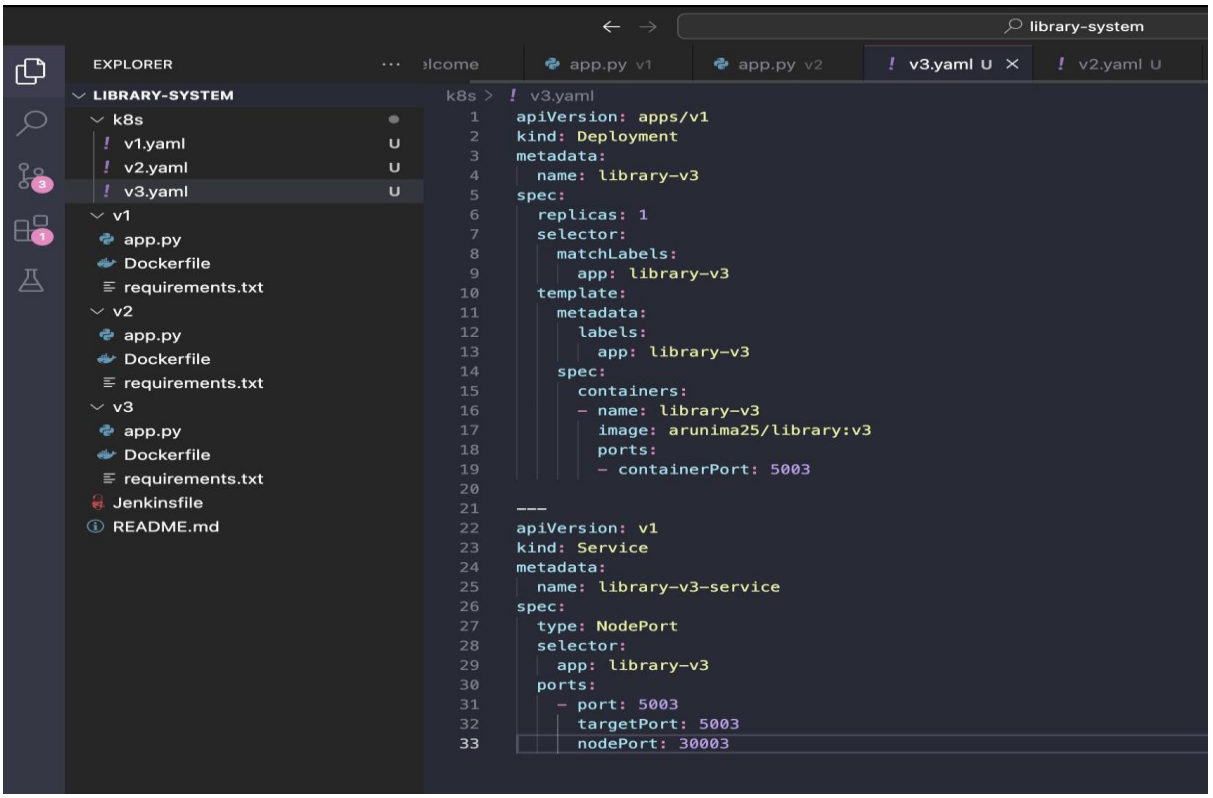


Figure 26: Kubernetes Configuration for v3

```
arunimasanjeev@Arunimas-MacBook-Air k8s % kubectl get pods
[kubectl get services
NAME                                      READY   STATUS             RESTARTS   AGE
library-v1-7c985dbc65-tgk74               1/1     Running            0          3m1s
library-v2-865f5dffc-sbxp8                1/1     Running            0          6s
library-v3-db7f68bcc-sbspr                1/1     Running            0          2m48s
staticwebapp-deployment-6b8f9f5966-bphjc  0/1     ImagePullBackOff   0          171d
staticwebapp-deployment-f855f9cd-psr5j    0/1     Pending            0          171d
```

Figure 27: Running pods

```
NAME                  TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)          AGE
kubernetes            ClusterIP   10.96.0.1        <none>        443/TCP          172d
library-v1-service    NodePort    10.108.121.218   <none>        5001:30001/TCP   3m1s
library-v2-service    NodePort    10.104.174.130   <none>        5002:30002/TCP   6s
library-v3-service    NodePort    10.105.109.185   <none>        5003:30003/TCP   2m48s
staticwebapp-service  NodePort    10.109.229.171   <none>        80:30080/TCP     171d
```

Figure 28: Kubernetes Services and ports

FINAL OUTPUT

The application was successfully deployed and accessed through different ports:

- v1 → http://localhost:30001/books

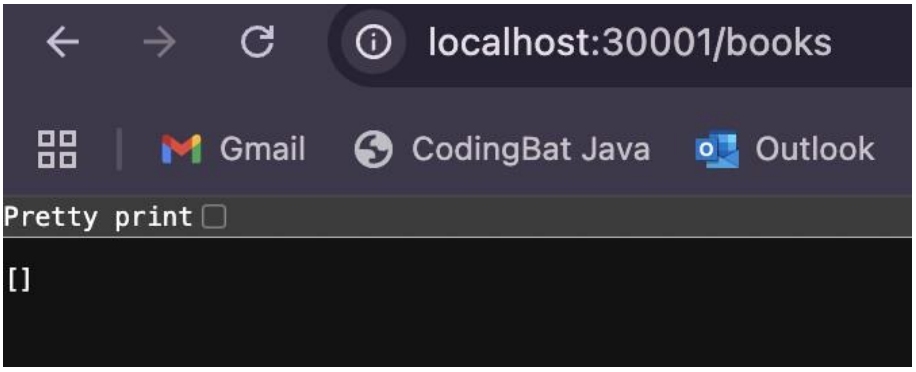


Figure 29: Output of Version 1 application accessed via NodePort 30001 using /books endpoint

- v2 → http://localhost:30002/books

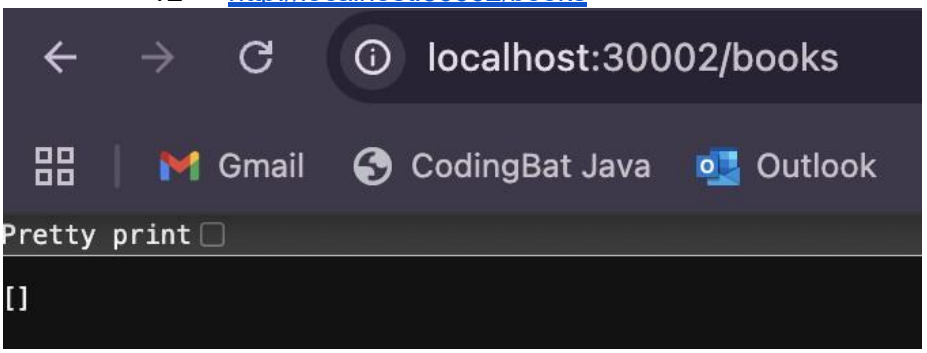


Figure 30: Output of Version 2 application accessed via NodePort 30002 using /books endpoint

- v3 → http://localhost:30003/books

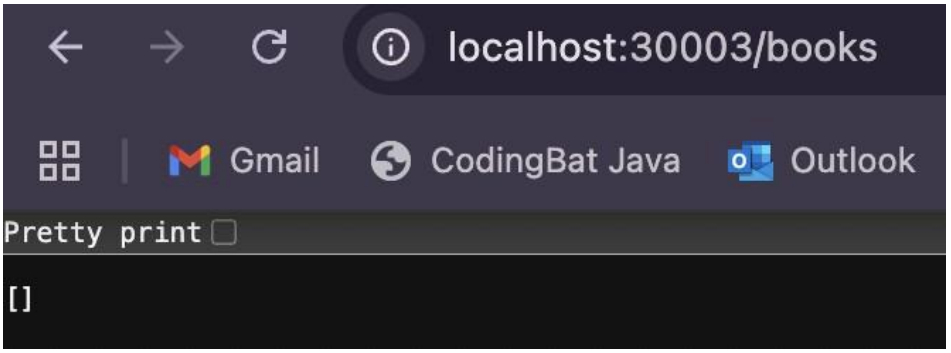


Figure 31: Output of Version 3 application accessed via NodePort 30003 using /books endpoint

The successful deployment and accessibility of all three versions through different ports demonstrate the effectiveness of the implemented CI/CD pipeline and container orchestration. Each version operates independently within Kubernetes, validating proper deployment, service exposure, and system scalability.

CONCLUSION

This project successfully demonstrated the implementation of a complete DevOps pipeline, integrating development, containerization, automation, and deployment. By developing multiple versions of the application, the project showcased incremental feature enhancement and modular design. The use of Docker ensured consistency across environments, while Jenkins automated the build and deployment process. Kubernetes enabled efficient orchestration and scalability of the application. Overall, this project provided practical insights into modern DevOps practices, highlighting the importance of automation, containerization, and orchestration in building scalable and efficient software systems.